

a rudimentary form of garbage collection. And most importantly, it generates the body of the signal functions.

Part 3: The basic data types

Now that you know Qt messes with C++ so much that it's unfair to call it C++ any more, you might guess that Qt has its own set of basic and complex data types that you're supposed to use instead of what's in the standard library. And you'd be right. While Qt doesn't break C++ (standards-compliant C++ code will always work in Qt), it does define its own data types that make programming with C++ so much easier. In particular, its types for strings and collections are a lot easier to use than the C++ standards library and the STL.

But let's start with the very basics—the integers, if you will. You have *qint8*, *qint16*, *qint32* and *qint64* for signed 1-byte, 2-byte, 4-byte and 8-byte integers. You have *quint8*, *quint16*, *quint32* and *quint64* for unsigned 1-byte, 2-byte, 4-byte and 8-byte integers. You have *qintptr* for pointers, and *qptrdiff* for signed pointers, typically used for storing pointer deltas. You have a single *qreal* for floating point numbers, which is a *typedef* for *double* everywhere except on ARM, where it is a *typedef* to *float* for performance reasons. There's no *qbool*, so you'll be using the standard *bool* datatype for Boolean data.

Now there are the more complex data types. You'll be dealing with *QByteArray* a lot, which is basically a data type for keeping generic data that grows and shrinks dynamically, i.e., for creating buffers. It's very easy to change the endianness of data inside *QByteArrays*, so this is going to be invaluable in network programming.

The next data type you'll deal with often is the *QString*. As suggested by the name, *QStrings* are used to store strings. It's way better than the standard library strings, though, because it allows for very easy text-processing (if you've used Python's *split* and *join* functions on strings to split a string on a certain character and/or join different strings with a certain character, *QString* allows that).

You'll also need to know about the *QStringLiteral* macro. You'll have to use this macro if you want to pass literal strings in function arguments that expect *QString* arguments, like:

```
someFunctionCall(QStringLiteral("Long live Mr Potato!\n"));
```

Before we move on from the subject of strings, there's a *QStringList* too. A *QStringList* is created when you split *QStrings* on a character, and it behaves exactly like a *QList<QString>*. As the name suggests, this is just a list of *QStrings*.

And finally, Qt has its own set of template collections. I've already mentioned the *QList*, which is just a list (an array that can grow infinitely) of objects, and there's also a *QVector*, which again is very similar to the standards library's *std::vector*. The other two types of collections you're very likely to use is the *QMap*, which is basically just a name-value pair to be used as associative arrays and dictionaries,

and the *QSet*, which is like a set in mathematics—a collection of unique objects.

Build systems and QtCreator

You can code in Qt with Vim (or Emacs, if you swing that way), and compile using CMake or whatever build system you prefer, but if you're just getting started, you should simply install QtCreator.

There are a few reasons for this. First, QtCreator is great. The code-completion and automated refactoring capabilities rival those of Visual Studio. It's very well designed, and devotes a lot of screen real estate to the actual text editor, instead of filling the screen with toolbars. And it looks good—the incredible features that it has are all hidden away but available with just a few mouse-clicks, so you don't feel restrained by your own IDE. It's also got in-built Git support.

Second, QtCreator takes care of the boilerplate for you, so that you don't have to spend time writing build scripts and the like. It integrates with *qmake*, Qt's own build tool, and for all the flak it receives, *qmake* is actually pretty awesome. So far, I've used QtCreator to create a piece of software and uploaded it to GitHub; and then written a *PKGBUILD* for Arch Linux that grabs the code off of GitHub, builds it and creates a complete package—one that installs binaries, and supports files in */share* and configuration files inside */etc*. Because QtCreator generates *qmake* project files, you don't need it to build these files; you simply need to run *qmake* instead of *./configure* to generate the make files and then just *make && sudo make install* as usual. You'll need *qmake* to be installed if you have the Qt-Core development package installed, so it's not an additional dependency (unlike CMake). And *qmake* project files are actually easy to read.

Due to space constraints, writing build scripts and *CMakeFiles* are out of the scope of this article (because of the MOC and other steps you need to take to set up the build environment, it's pretty complicated). I'm going to explain how to use QtCreator only, and build stuff with *qmake*. Don't worry, it's easy and professionals do use QtCreator. You're not sacrificing the powerful way for the easy way.

In the next article

Now that you know a little bit about the theory behind Qt's awesome architecture, you're ready to write some code. In the next article, we'll write a network server application that sends out a random fortune cookie to anyone who connects to it. And as you will see, it is going to be too easy for words. 

By: Boudhayan Gupta

Describing himself as a 'retard by choice', the author believes that madness is a cure-all for whatever is wrong or right with society. A social media enthusiast, he can be reached at [BaloneyGeek](https://twitter.com/BaloneyGeek) on Twitter.